
Chat -GPT based HITL workflow

(Human In The Loop)

2026.06.05

WonJong Jeong

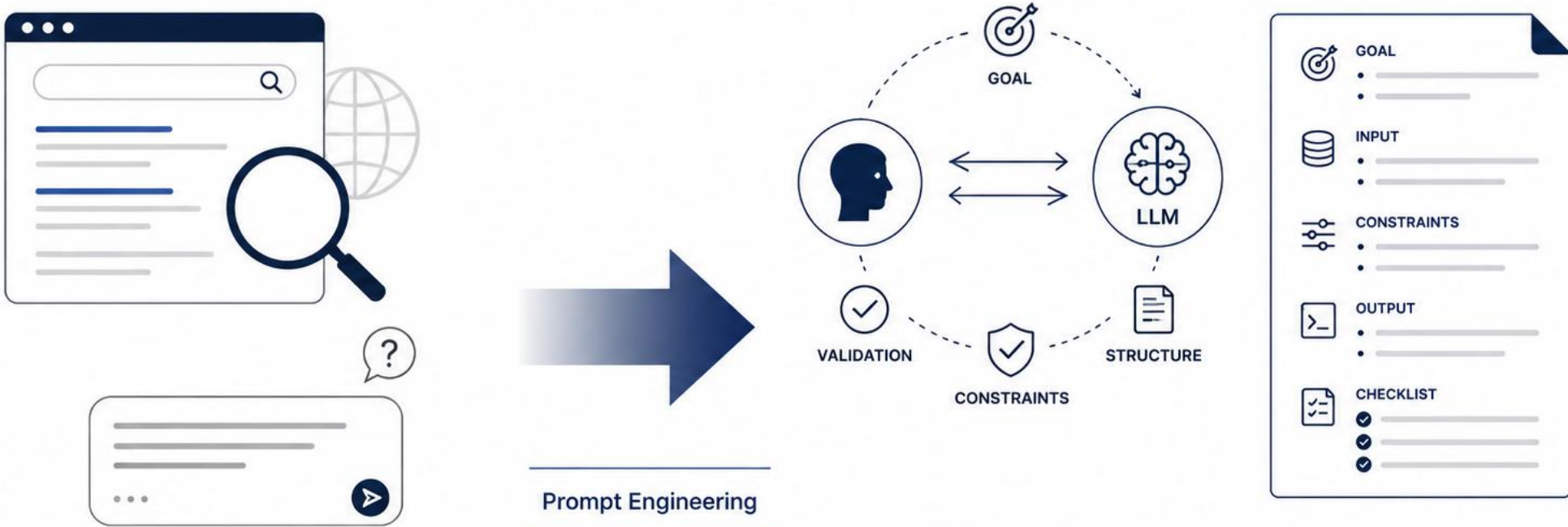
AiM⁴ Lab

1. Discussion with the LLM: From Idea to Structured Task
2. Prompt to Code: System Design Workflow (LLM serving chatbot)

Discussion with the LLM: From Idea to Structured Task

Search Engine to Working Partner

4



단순 검색·질문 도구로 활용

정보 탐색, 단편적 답변 중심



구조화된 프롬프트 기반 협업

목표 지향적 설계, 일관된 결과, 재현 가능성 향상

LLM의 출력 품질은 입력 구조에 따라 달라진다

목표 · 입력 · 제약 · 출력 · 검증을 명확히 구조화할수록 더 정확하고 일관된 결과를 얻을 수 있다.

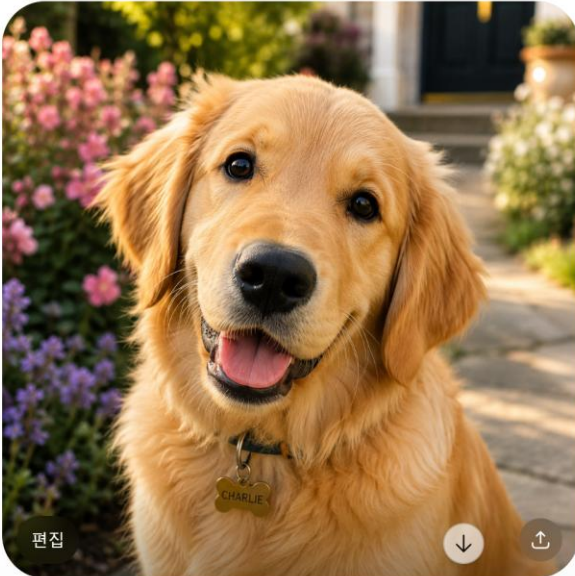
Search Engine to Working Partner

5

Input 영역 지정에 따른 원하는 결과물 생성 예시
(Chat GPT, GPT image 2)

개 사진 그려줘

Thought for 33s >

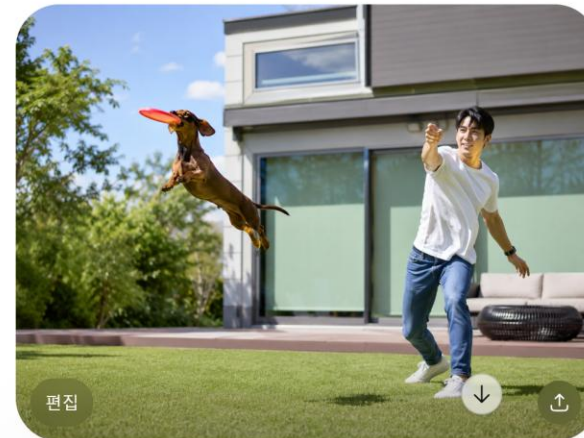


<Output with few prompt>

문제 구조화 및
필요한 정보 영역 정의

개 사진 그려줘.
견종 : 닥스훈트
자세 : 공중에서 원반을 잡은 상태
배경 : 마당이 있는 최신식 집, 마당쪽
등장인물 : 1)개 주인, 20대 남성, 건강함, 개와 놀아주고 있음
2)개 (위의 설명)
그림체 :실사풍

Thought for 39s >



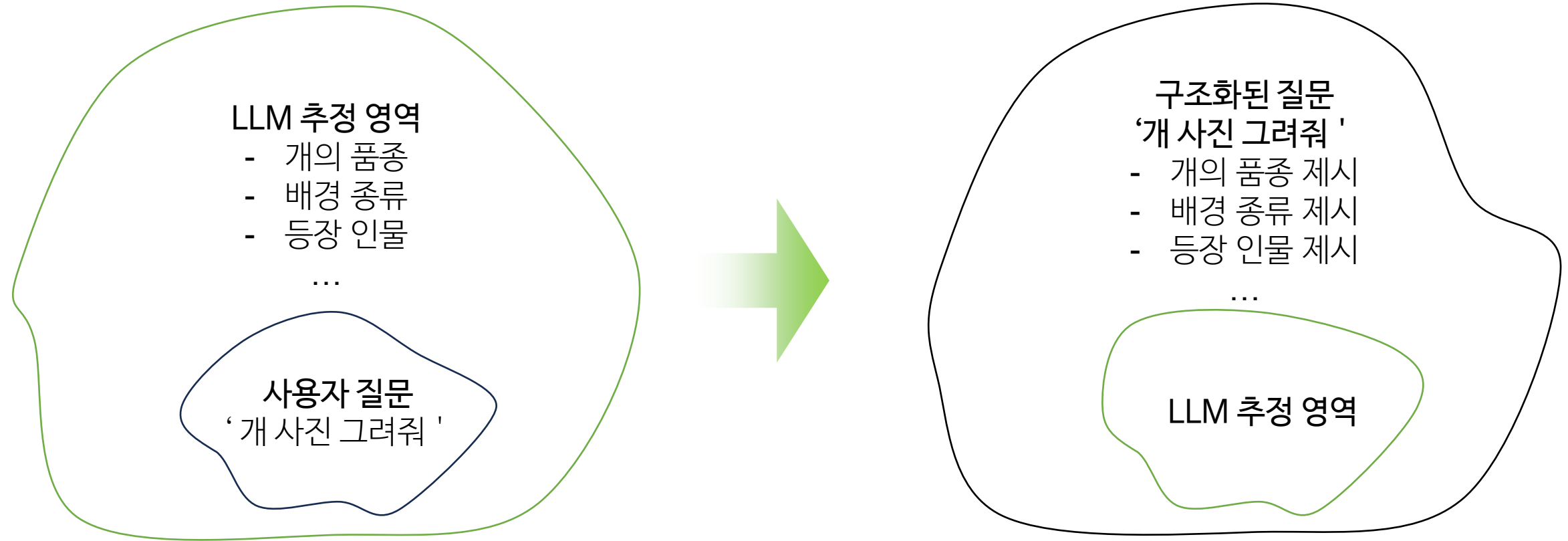
<Output with structured prompt>

Domain Knowledge에 따라 결과물 품질 결정

Search Engine to Working Partner

6

답변을 얻기 위한 질문 영역의 확장



LLM 추정 영역 감소 → Hallucination 방지 및 답변 고도화

Discussion with the LLM

7

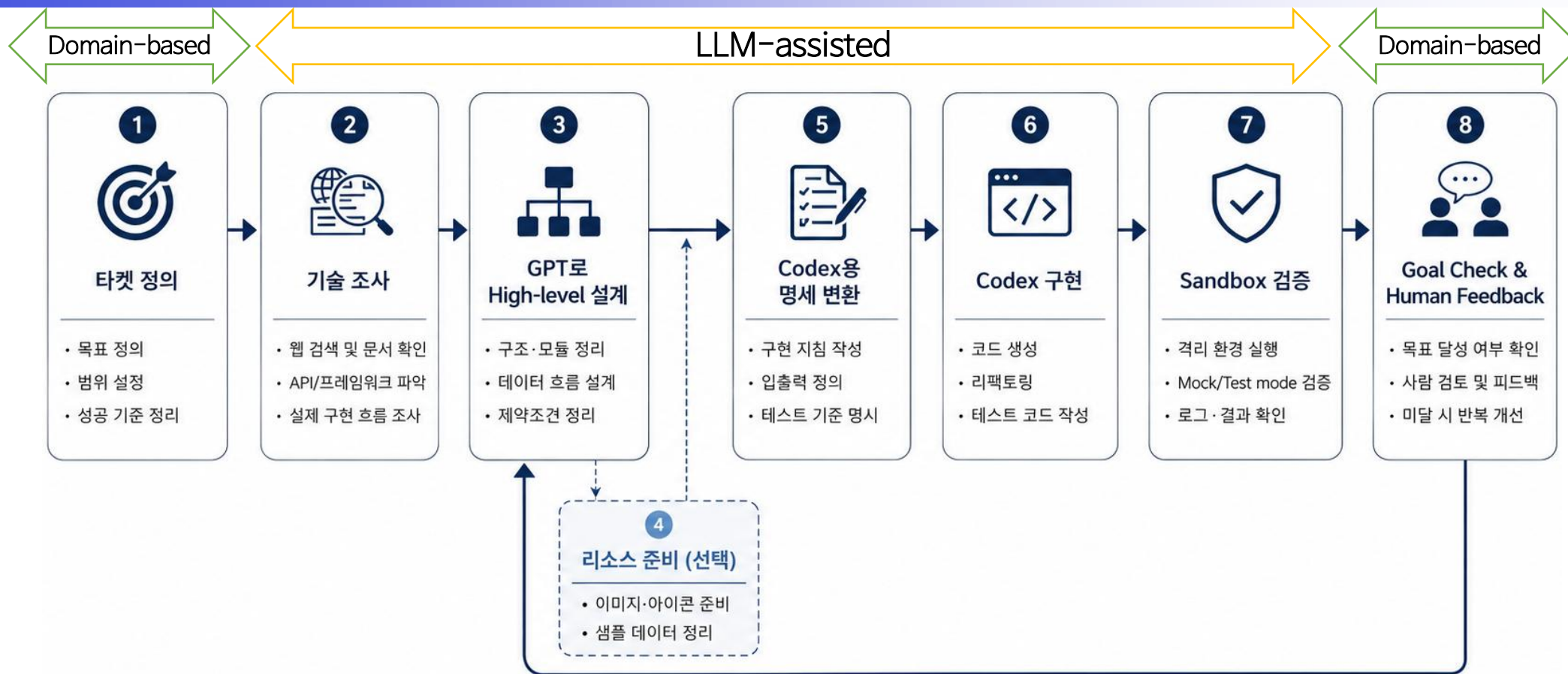


Take-home message

아이디어를 바로 코드로 바꾸지 말고, 먼저 구현 가능한 요구사항으로 바꿔라.

좋은 코드는 좋은 질문이 아니라, 좋은 요구사항에서 나온다.

Prompt to Code: System Design Workflow



좋은 코드는 좋은 아이디어만으로 나오지 않는다. 구조화된 명세와 검증 루프에서 나온다.

목표 정의 → 설계 → 구현 → 검증 → 피드백의 흐름이 중요하다.

Codex란?

OpenAI의 클라우드 기반 코드 생성·수정·실행 AI 에이전트
자연어 명세를 코드로 변환하고, 테스트·수정까지 자동 수행



코드 생성 및 수정

자연어 명세를 기반으로 코드 작성,
기존 코드 이해 및 리팩터링



안전한 격리 실행 환경

샌드박스 환경에서 코드 실행, 빌드, 테스트 수행
결과 및 로그 제공



반복적 개선 (Iterative Refinement)

테스트 결과/피드백을 반영하여
코드 수정 및 재실행

핵심 특징

- ✓ 자연어 → 실행 가능한 코드/산출물 자동 변환
- ✓ 파일/프로젝트 컨텍스트를 이해하고 작업 수행
- ✓ 테스트 실패 시 원인 분석 및 수정 제안/반영
- ✓ 다양한 언어 및 프레임워크 지원
- ✓ 인간 검토(Human-in-the-loop)와 결합된 안전한 자동화

동작 원리



Codex 역할

요구사항을 구현 가능한 코드로 변환하고, 테스트·검증을 통해 동작하는 산출물을 만듭니다.



활용 효과



개발 생산성 향상
반복 작업 자동화로
개발 속도 가속



정확도 및 일관성
명세 기반 구현으로
휴먼 에러 감소



안전한 자동화
격리된 환경에서 실행하여
안정성 확보



도메인 집중
엔지니어는 문제 정의와
검증에 집중



새로운 마법이 아니라, 기존 노하우를 구조화하는 과정

원리를 알고 쓰면 재현성과 활용 효율이 높아진다.



익숙한 현상도 공학적 원리가 있다



핵심 포인트

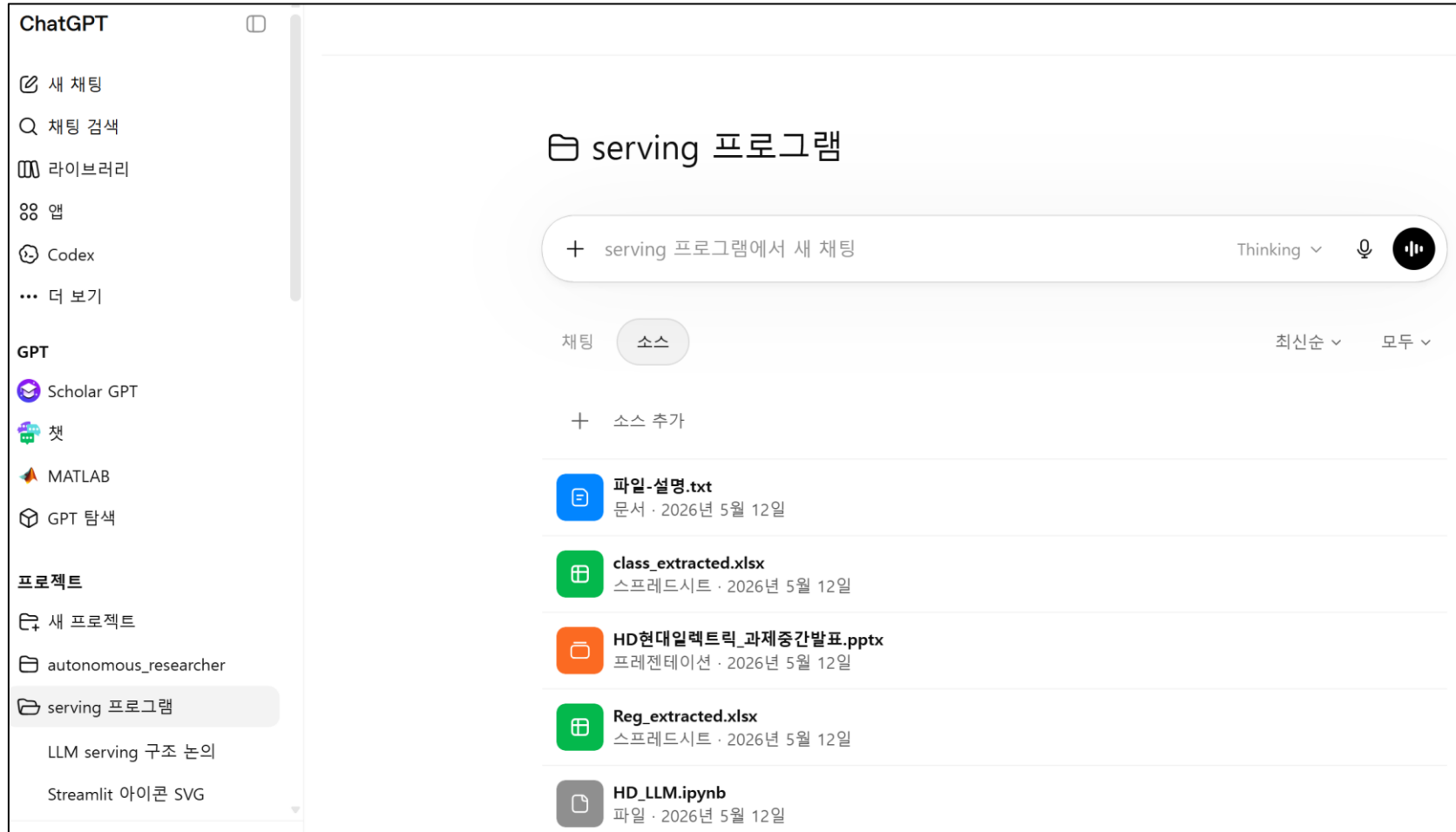
- ✓ 많이 말하는 것보다 필요한 context를 선별하는 것이 중요하다
- ✓ 익숙한 도구도 원리를 알면 더 효율적으로 활용할 수 있다

⚙️ 공학적 해석

- LLM은 현재 질문만이 아니라 이전 대화 맥락(context)을 함께 처리한다
- Context가 길어질수록 처리해야 할 **token 수**가 증가한다
- Transformer 기반 **attention** 계산과 **메모리 사용량**이 함께 커진다
- 따라서 **latency**와 **computational cost**가 증가한다

GPT를 통한 High level 설계 (머신러닝 프로그램의 LLM 서빙)

13



프로젝트 생성

→ 소스 파일 추가

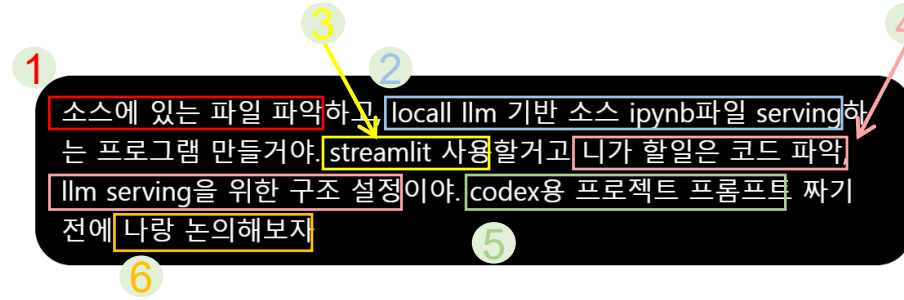
- 프로젝트 개요 ppt
- 코드 파일
- 실제 다룰 데이터 내용 등

*유출될 위험이 있는 민감한 정보 사전 제거 필요

→ 소스 파일 설명 지침 txt 추가

- 각 파일 내용 간단 설명

📁 serving 프로그램



먼저 업로드된 파일 역할과 노트북 코드 흐름을 파악하고, 입출력·모델·의존성을 기준으로 Streamlit+local LLM 서빙 구조를 논의 가능한 모듈 단위로 정리하겠습니다.

현재 노트북은 PyCaret 기반 분류/회귀 모델 비교·평가용 실험 코드이며, 저장된 모델을 로드해 예측하는 서빙 코드는 아직 없습니다. 따라서 전처리-학습-모델저장-추론-LLM 설명 계층으로 분리하는 방향이 안전합니다.

Thought for 5m 35s >

좋아. 지금 상태 기준으로 보면 이 프로젝트는 ***노트북 탐색 코드 → 재현 가능한 학습/추론 파이프라인 → Streamlit UI → local LLM 기반 질의/설명 레이어***로 재구조화해야 합니다.

1. 내가 확인한 소스 파일 역할

업로드된 설명 파일 기준으로 구조는 단순합니다: 엑셀은 데이터, PPT는 프로젝트 개요, ipynb는 핵심 코드입니다. 📁 파일-설명

파일	역할	현재 판단
HD_LLM.ipynb	↓ 코드	PyCaret 기반 분류/회귀 탐색 코드. 아직 serving 구조 없음

프롬프트 분석

- 1) 소스에 있는 파일 파악
 - RAG(참조)할 내용 영역 지정
- 2) Local LLM 기반 소스 IPYNB 파일 Serving 하는 프로그램 만들거야
 - Long term goal 지정
- 3) Streamlit 사용할거고
 - Constraint 부여(챗봇 개발 라이브러리)
- 4) 니가 할일은 코드 파악, LLM 서빙을 위한 구조 설정이야
 - Short term goal 지정
- 5) Codex용 프롬프트 작성
 - Middle term goal 지정
- 6) ~전에 나랑 논의 해보자
 - Human in the loop trigger

📁 serving 프로그램

2차에서 SHAP 고도화, 3차에서 이미지/CNN/멀티모달로 가는 게 안정적입니다.

11. 지금 우리가 정해야 할 핵심 결정

가장 먼저 하나만 정하면 됩니다.

LLM의 역할을 “프로젝트/코드 설명용 챗봇”으로 둘지, 아니면 “예측 tool까지 호출하는 agent”로 둘지입니다.

내 추천은 두 번째입니다. 즉, LLM은 답변만 하는 챗봇이 아니라, 사용자가 자연어로 조건을 입력하면 feature 누락을 체크하고, deterministic ML prediction tool을 호출한 뒤, 그 결과를 설계자 친화적으로 설명하는 구조가 좋습니다.

📄 👍 🗨️ 📄 🔄 ... 🏠 🤖 출처

1

데이터는 엑셀 데이터가 맞고, 툴콜링으로 하면 되겠다.

맞습니다. 그러면 LLM은 답변 생성기가 아니라 엑셀 기반 ML 예측 도구를 호출하는 오케스트레이터로 두고, 구조를 tool-calling 중심으로 정리하겠습니다.

2

streamlit gui에 데이터 업로드 공간을 만들고 엑셀 파일도 넣을수 있게 하면 되잖아



프롬프트 분석

1) 데이터는 엑셀 데이터가 맞고, 툴콜링으로 하면 되겠다

- LLM의 제안 사항 사용자가 읽고 답변
- Domain knowledge 필요
[ex) tool-calling 기능, 그 기능을 사용할 수 있는 LLM 모델 등등..]

2) Streamlit GUI에 데이터 업로드 공간을 만들고 엑셀 파일도 넣을수 있게 하면 되잖아

- Follow up 기능을 활용한 컨텍스트 윈도우 축소

📁 serving 프로그램

1

그럼 codex용 프롬프트 txt파일로 만들어줘

Codex가 바로 구현할 수 있도록 목적, 아키텍처, 금지사항, 파일 구조, 구현 요구사항, 검증 기준을 포함한 txt 프롬프트로 정리하겠습니다.

업로드 UI와 로컬 LLM tool calling도 명시하겠습니다. Streamlit은 `st.file_uploader`, `st.chat_input`, `st.cache_resource` 기반, Ollama는 교체 가능한 adapter로 넣겠습니다.

Thought for 4m 33s >

완성했습니다. 아래 txt 파일을 Codex 프로젝트 프롬프트로 바로 붙여넣으면 됩니다.

2

Codex용 프로젝트 프롬프트 다운로드

파일에는 **Excel 업로드 GUI, classification/regression preprocessing, model artifact 저장, Streamlit serving, local LLM/Ollama tool calling, schema 검증, batch prediction, SHAP/fallback explanation, 테스트 기준까지** 포함했습니다.

업로드된 자료 역할은 “엑셀=데이터, PPT=프로젝트 개요, ipynb=핵심 코드”라는 설명을 반영했고

📄 파일-설명, PPT의 `Result == 1.0 → 차단성공`, 나머지 `차단실패` 이진분류 정의와 GBC/SHAP 기반 해석 방향도 반영했습니다. 📄 HD현대일렉트릭_과제중간발표 Streamlit 업로드/챗/캐시 구조는 공식 `st.file_uploader`, `st.chat_input`, `st.cache_resource` 기준으로 넣었고, Ollama는 공식 tool calling 및 `/api/chat` 구조를 기준으로 적었습니다. Streamlit 문서 >

+ 무엇이든 물어보세요

Pro ▾



Codex Project Prompt: HD현대일렉트릭 수치형 데이터 기반 ML + Local LLM Serving Program

당신은 이 저장소를 정리하고 구현하는 Codex 엔지니어이다. 목표는 기존 'HD_LLM.ipynb'의 분류/회귀 ML 로직을 LLM 기반 serving 프로그램을 만드는 것이다.

이 프롬프트의 요구사항을 우선순위 높게 따른다. 구현 중 불명확한 부분이 있으면 합리적인 MVP 결정을 내리고

0. 현재 업로드/소스 파일 역할

현재 프로젝트의 입력 파일은 다음과 같다.

- 'HD_LLM.ipynb': 핵심 notebook 코드. PyCaret 기반 classification/regression 모델 탐색 코드가 들어 있다.
- 'class_extracted.xlsx': classification용 수치형 Excel 데이터.
- 'Reg_extracted.xlsx': regression용 수치형 Excel 데이터.
- 'HD현대일렉트릭_과제중간발표.pptx': 프로젝트 개요, 문제 정의, 성능 해석, SHAP/이미지 DB 계획 등이 담긴 발표자료
- '파일-설명.txt': 파일 역할 설명. 엑셀은 데이터, PPT는 프로젝트 개요, ipynb는 핵심 코드라고 설명한다.

발표자료 기준 프로젝트 목표는 형상/시험 변수 기반 고압차단기 성능 예측 모델을 만드는 것이다. 수치형 DB로 하며, 'Result == 1.0'이면 차단성공 Class 1, 그 외는 차단실패 Class 0으로 정의한다.

1. 기존 notebook에서 반드시 반영할 로직

1.1 Classification notebook logic

기존 notebook의 classification 로직은 다음과 같다.

```
python
import numpy as np
import pandas as pd
from pycaret.classification import *
```

```
df = pd.read_excel('C:/Users/user/class_extracted.xlsx')
df = df.dropna()
df = df.reset_index(drop=True)
```

```
label = np.where(df['Result'] == 1.0, int(1), int(0))
label = label.reshape(-1, 1)
```

```
df['label'] = label
df = df.drop(['Result', 'TRVmax[kV]'], axis=1)
```

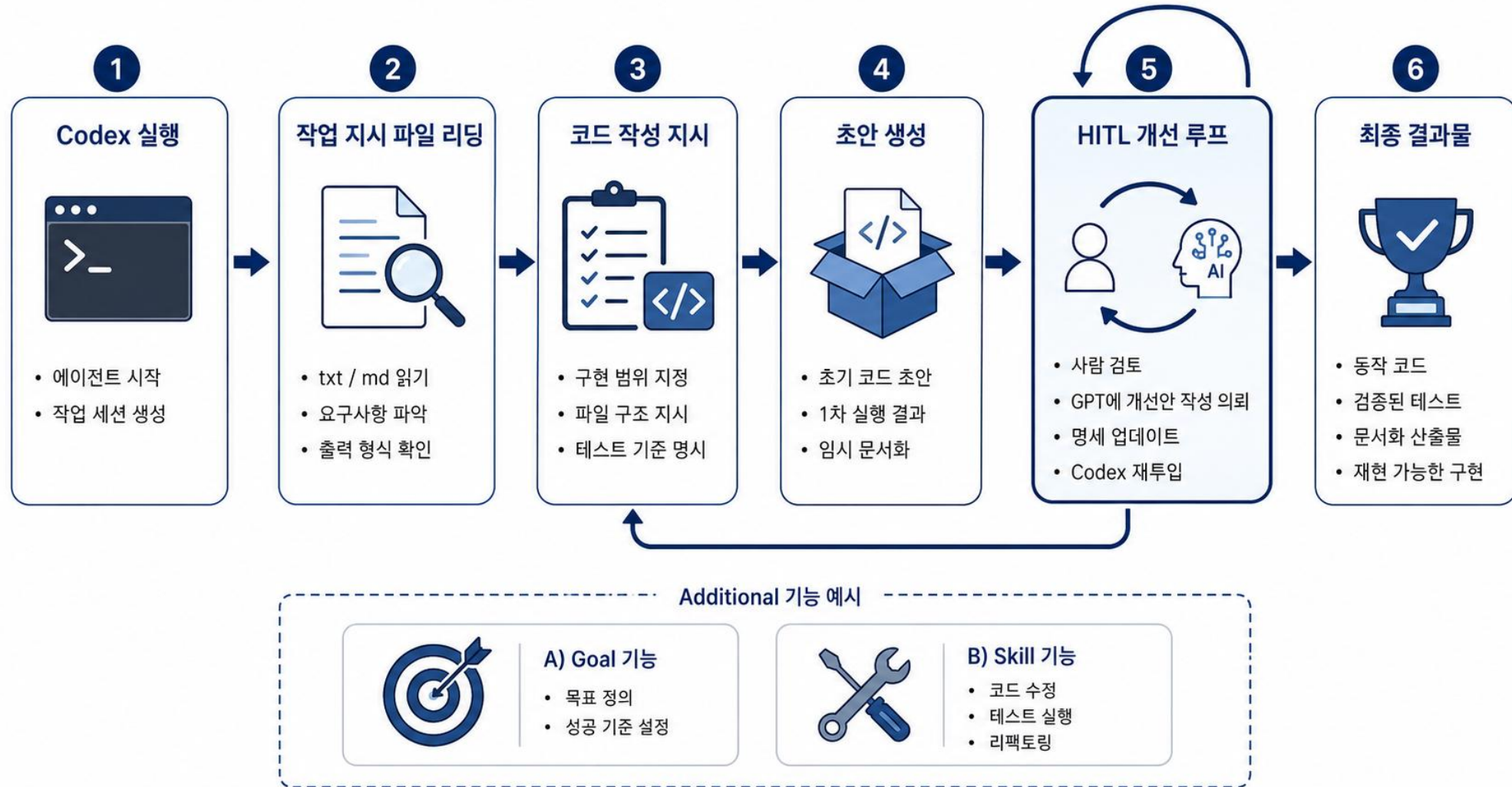
```
setup = setup(data=df, target='label', session_id=0, train_size=0.9)
```

줄 28, 열 1 21,143자

일반 텍스트

CODEX Workflow

17

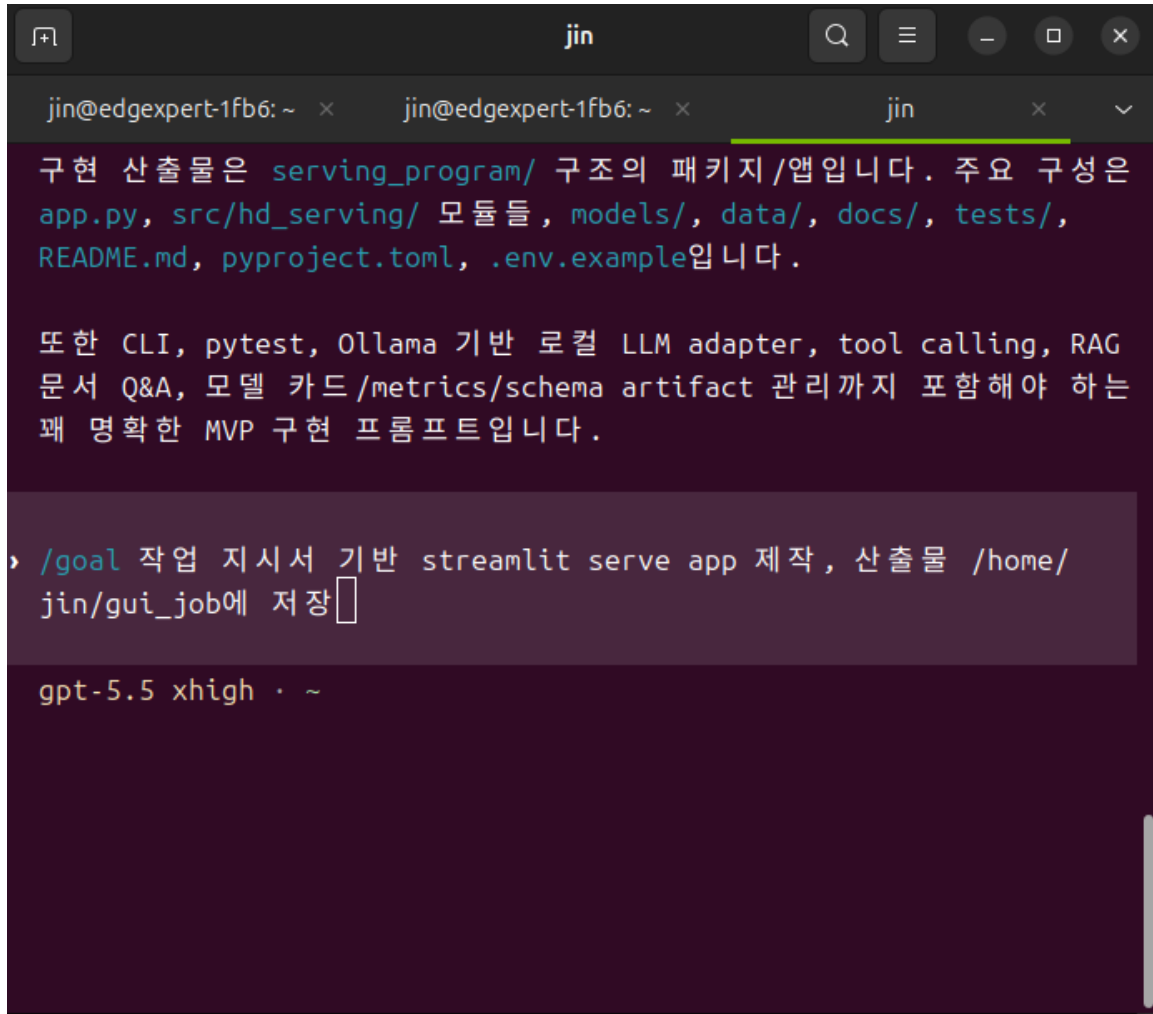


```
jina@edgexpert-1fb6: ~  
jina@edgexpert-1fb6: ~  
jina@edgexpert-1fb6: ~$ codex  
⚠Codex's Linux sandbox uses bubblewrap and needs access to create  
user namespaces.  
  
>_ OpenAI Codex (v0.136.0)  
  
model: gpt-5.5 xhigh /model to change  
directory: ~  
  
Tip: Switch models or reasoning effort quickly with /model.  
  
> /home/jin/Desktop/codex_project_prompt_hd_serving.txt 파일 내용  
파악해줘  
  
gpt-5.5 xhigh · ~
```

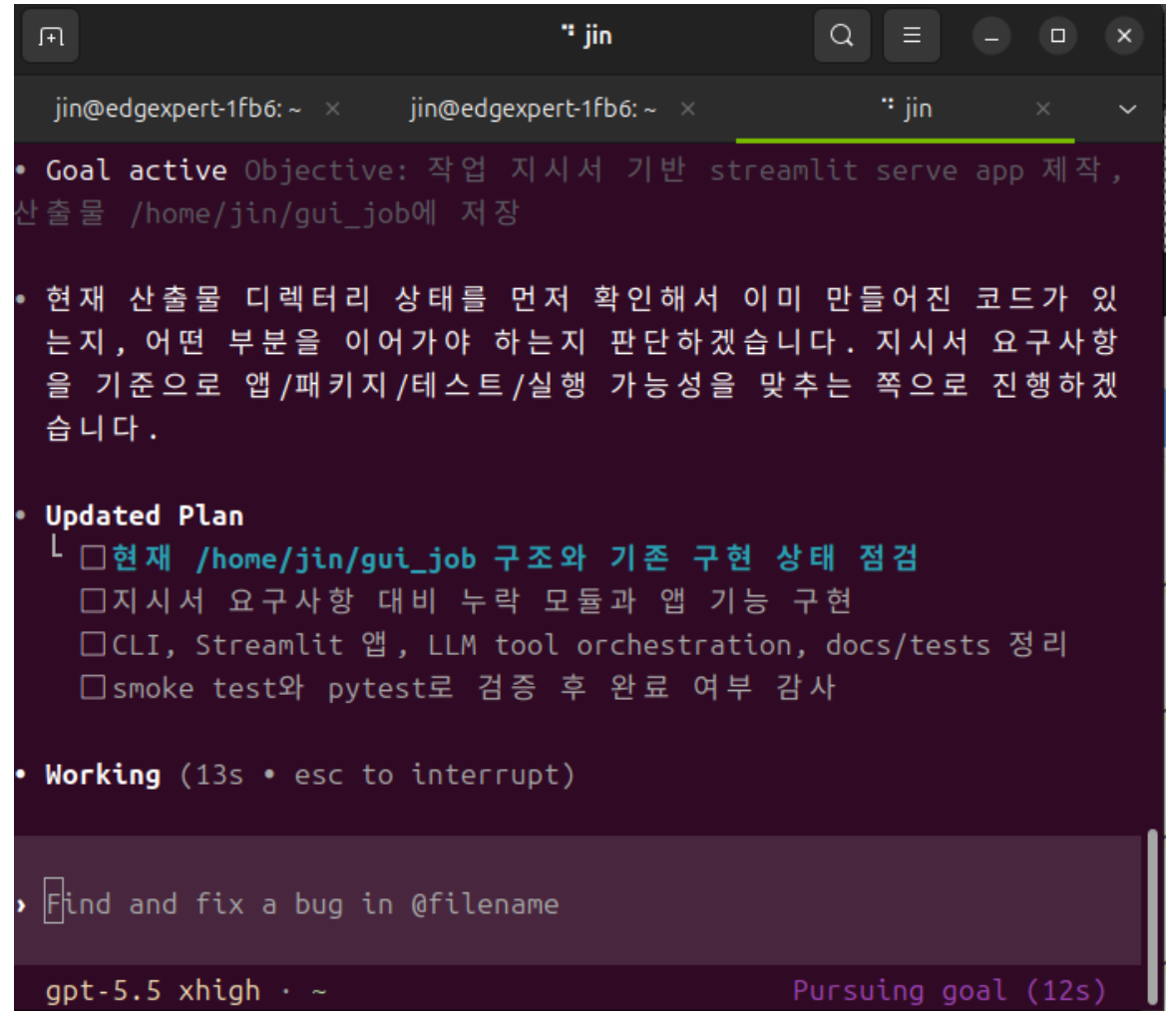
작업 지시 파일 리딩

```
jina@edgexpert-1fb6: ~  
jina@edgexpert-1fb6: ~  
jina@edgexpert-1fb6: ~  
• 파악했습니다. Desktop/codex_project_prompt_hd_serving.txt는 HD현대  
일렉트릭 수치형 데이터 기반 ML + Local LLM Serving 프로그램 구현  
지시서입니다.  
  
핵심 목표는 기존 HD_LLM.ipynb의 PyCaret 기반 분류/회귀 로직을 재현  
가능한 Python 패키지로 분리하고, Streamlit 앱에서 Excel 업로드, 데  
이터 검증, 모델 학습, 예측, 로컬 LLM 질의응답까지 처리하는 서빙 프  
로그램을 만드는 것입니다.  
  
가장 중요한 요구사항은 다음과 같습니다.  
  
- 분류는 Result == 1.0만 성공 label=1, 나머지는 0.  
- 분류 feature에서는 Result, TRVmax[kV]를 반드시 제외.  
- 회귀 target은 TRVmax[kV].  
- 회귀 feature에서는 Time, Result, CZM, Test, TRVmax[kV]를 반드시  
제외.  
  
- notebook의 절대경로는 사용하지 않고 project-relative path 또는
```

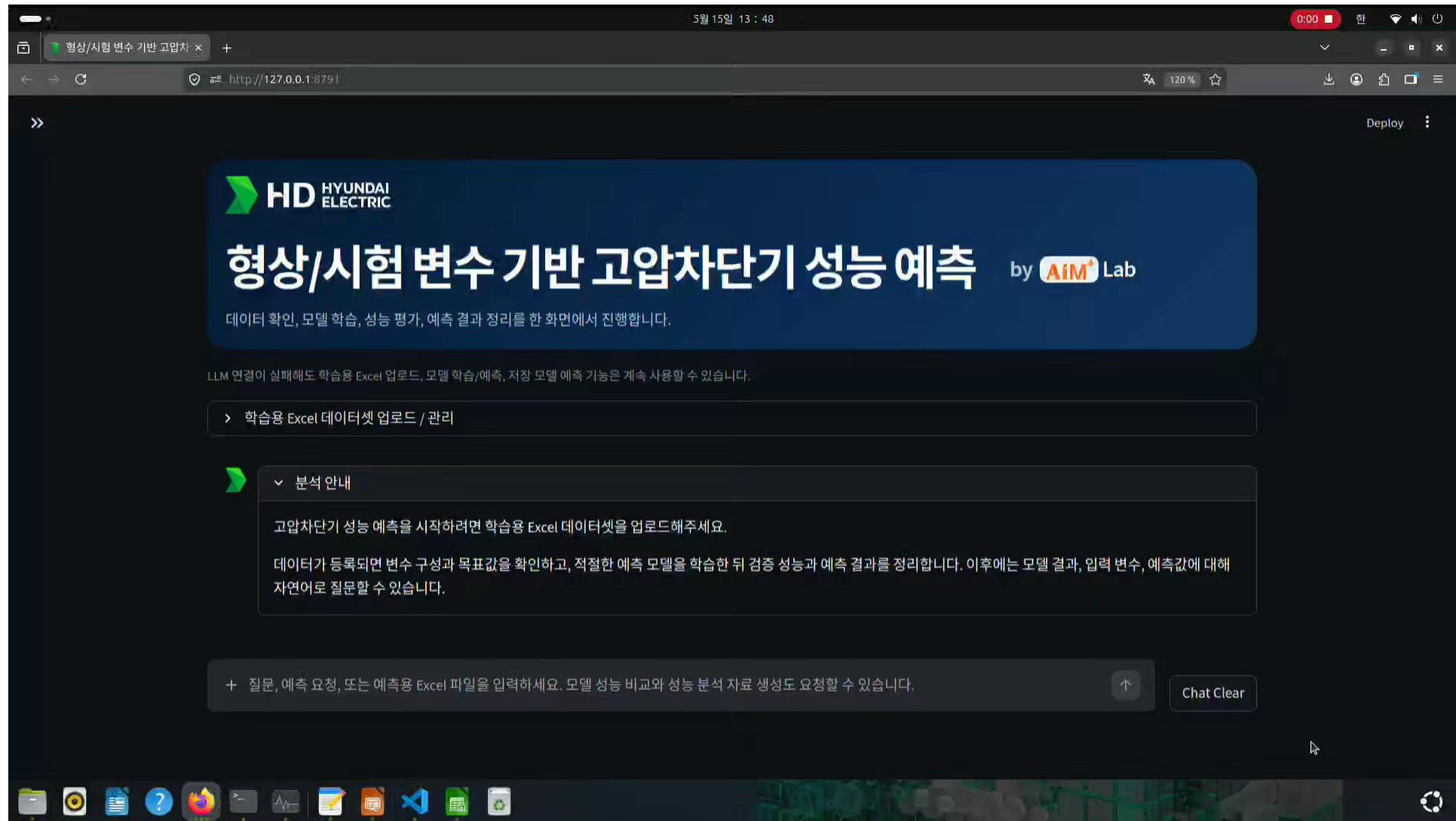
CODEX가 내용 파악

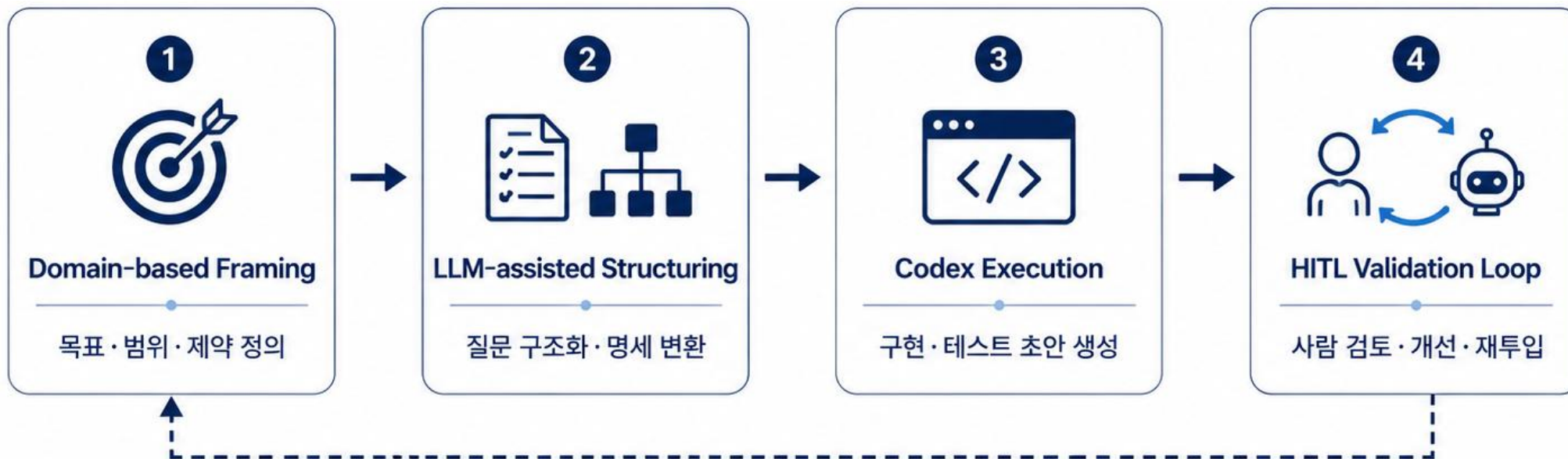


GOAL 기능을 통한 구현 지시



CODEX가 허용 가능한 Context만큼 planning
 -> term 별 작업 수행 (목표 반복시까지 반복)
 -> 사람이 결과물 보고 재작업 or 완료





좋은 결과물은 구조화된 요구사항과 검증 루프에서 나온다



재현 가능성 향상



개발 효율 향상



신뢰도 높은 결과

Thank you for listening
